

Alexandria University
Faculty of Engineering
Computer and Communication Program

CC484 Pattern Recognition

Assignment Two: Network Anomaly Detection



Project Link: [Colab Notebook](#)

Name	ID
Omar Walied Mohamed	7058
Hend Khaled AbdelHamid M. Aly	6986
Habiba Mohamed Hefny	6939

Table of Contents

Introduction to the problem statement.....	2
Problem Description.....	2
Code Explanation and Screenshots	4
1. Analyzing and Preprocessing Dataset	4
2. Kmeans Clustering	6
Algorithm.....	6
K = 7	8
K = 15	9
K = 23	10
K = 31	11
K = 45	12
3. Normalized Cut Clustering.....	13
4. Evaluation.....	14
Kmeans.....	14
Normalized Cut	21
5. New Clustering (DBSCAN)	22
Algorithm.....	22
Evaluation.....	25

- **What is Network Anomaly Detection?**

Network anomaly detection in machine learning refers to the process of identifying unusual or unexpected patterns or behavior in network traffic that may indicate the presence of security threats such as malware infections, cyber attacks, or other malicious activities.

Anomaly detection algorithms analyze network traffic data to identify deviations from normal patterns and flag suspicious activities that could indicate potential security threats. This is done by training a machine learning model on a large dataset of network traffic, identifying patterns and behaviors that are considered normal, and then using this model to detect deviations from the norm.

There are various techniques used for network anomaly detection, including statistical analysis, clustering, and classification algorithms. These techniques help to identify anomalies in different types of network traffic, such as packets, flows, or sessions.

Network anomaly detection is an important tool for enhancing the security of networks, allowing organizations to quickly detect and respond to potential threats before they can cause significant damage.

- **Problem Description:**

First. downloaded the KDD cup 1999 dataset from kaggle website.

Second. Imported the needed libraries.

Third. Import the 10 percent data and assign it to 'train', then provide some basic statistical information about the data in the DataFrame, such as the count, mean, standard deviation, and quartiles of each column.

Fourth. Assign the 'corrected.gz' data to 'test', then provide basic statistical information about the data in the DataFrame, such as the count, mean, standard deviation, and quartiles of each column. This

information can be used to compare the test dataset with the training dataset and check for any significant differences between the two.

Fifth. Preprocess the data for training and testing datasets where we create lists of the service, flag, label columns from both training and testing datasets then assign a unique integer value to each item in these lists using numpy's zero's function.

Sixth. Replace the string values in the "protocol_type", "service", "flag", and "label" columns of both the training and testing datasets with their corresponding integer values.

Seventh. Shuffle the rows of train DataFrame randomly by calling the sample function with frac=1, avoid any order or bias in the training data, remove first column of the shuffled train dataframe (old index column).

Eighth. Perform our implementation of Kmeans algorithm several times, each time with a different number of clusters. Where we'll change the k of the Kmeans algorithm between {7, 15, 23, 31, 45} to produce different clusters.

Ninth. Perform Normalized Cut Clustering that we implemented where we used 42 as the random seed across all experiments, Split the training dataset used, apply the normalized cut algorithm to the preprocessed data to cluster it into 23 clusters, rerun the experiments on kmeans when k=23.

Tenth. Evaluated the models after using kmeans clustering using precision, recall, f1 score, and conditional entropy. We did the same process after performing the normalized cut clustering.

Eleventh. Implemented a new clustering algorithm which is DBSCAN and evaluated it using precision, recall, f1 score, and conditional entropy.

```
[93] from google.colab import drive
drive.mount('/content/drive/')

Drive already mounted at /content/drive/; to attempt to forcibly remount, call drive.mount("/content/drive/", force_remount=True).

[94] %cd /content/drive/MyDrive/PR/assignment2

/content/drive/MyDrive/PR/assignment2

[ ] !unzip archive.zip -d '/content/drive/MyDrive/PR/assignment2'

Archive: archive.zip
replace /content/drive/MyDrive/PR/assignment2/corrected.gz? [y]es, [n]o, [A]ll, [N]one, [r]ename: N

import pandas as pd
from sklearn.metrics.pairwise import pairwise_distances
import numpy as np
from time import time
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.cluster import KMeans
import random
from sklearn.utils import check_random_state
from sklearn.preprocessing import normalize
from scipy.sparse.linalg import eig
from sklearn.metrics.pairwise import rbf_kernel
import random
```

```
#read the file containing train data
col_names = ["duration", "protocol_type", "service", "flag", "src_bytes",
             "dst_bytes", "land", "wrong_fragment", "urgent", "hot", "num_failed_logins",
             "logged_in", "num_compromised", "root_shell", "su_attempted", "num_root",
             "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
             "is_host_login", "is_guest_login", "count", "srv_count", "error_rate",
             "srv_error_rate", "error_rate", "srv_error_rate", "same_srv_rate",
             "diff_srv_rate", "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
             "dst_host_same_srv_rate", "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
             "dst_host_srv_diff_host_rate", "dst_host_error_rate", "dst_host_srv_error_rate",
             "dst_host_rerror_rate", "dst_host_srv_rerror_rate", "label"]
train = pd.read_csv("/content/drive/MyDrive/PR/assignment2/kddcup.data_10_percent.gz", compression='gzip', header=None, names = col_names)
train.describe()
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	dst_host_count	dst
count	494021.000000	4.940210e+05	4.940210e+05	494021.000000	494021.000000	494021.000000	494021.000000	494021.000000	494021.000000	494021.000000	...	494021.000000	
mean	47.979302	3.025610e+03	8.685324e+02	0.000045	0.006433	0.000014	0.034519	0.000152	0.148247	0.010212	...	232.470778	
std	707.746472	9.882181e+05	3.304000e+04	0.006673	0.134805	0.005510	0.782103	0.015520	0.355345	1.798326	...	64.745380	
min	0.000000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	
25%	0.000000	4.500000e+01	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	255.000000	
50%	0.000000	5.200000e+02	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	255.000000	
75%	0.000000	1.032000e+03	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	255.000000	
max	58329.000000	6.933756e+08	5.155468e+06	1.000000	3.000000	3.000000	30.000000	5.000000	1.000000	884.000000	...	255.000000	

8 rows x 38 columns

```
[97] #read the file containing test data
test = pd.read_csv("/content/drive/MyDrive/PR/assignment2/corrected.gz", compression='gzip', header=None, names = col_names)
test.describe()
```

	duration	src_bytes	dst_bytes	land	wrong_fragment	urgent	hot	num_failed_logins	logged_in	num_compromised	...	dst_host_count	dst
count	311029.000000	3.110290e+05	3.110290e+05	311029.000000	311029.000000	311029.000000	311029.000000	311029.000000	311029.000000	311029.000000	...	311029.000000	
mean	17.902736	1.731702e+03	7.479937e+02	0.000029	0.000762	0.000051	0.014677	0.002363	0.172476	0.011243	...	235.282681	
std	407.644400	1.276567e+05	1.612018e+04	0.005379	0.040367	0.009821	0.312068	0.049990	0.377794	1.958325	...	60.913298	
min	0.000000	0.000000e+00	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	0.000000	
25%	0.000000	1.050000e+02	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	255.000000	
50%	0.000000	5.200000e+02	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	255.000000	
75%	0.000000	1.032000e+03	0.000000e+00	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	...	255.000000	
max	57715.000000	6.282565e+07	5.203179e+06	1.000000	3.000000	3.000000	101.000000	4.000000	1.000000	796.000000	...	255.000000	

8 rows x 38 columns

```

▶ #preprocessing data for training and testing datasets

#create lists of the service, flag, label columns from both training and testing datasets
#then assign a unique integer value to each item in these lists using numpy's zeros function
services=pd.concat([test['service'],train['service']]).unique()
services_num=np.zeros(len(services))
for i in range (len(services)):
    services_num[i]=i

flags=pd.concat([test['flag'],train['flag']]).unique()
flags_num=np.zeros(len(flags))

for i in range(len(flags)):
    flags_num[i]=i

labels=pd.concat([test['label'],train['label']]).unique()
labels_num=np.zeros(len(labels))
for i in range (len(labels)):
    labels_num[i]=i

#replace the string values in the "protocol_type", "service", "flag", and "label" columns
#of both the training and testing datasets with their corresponding integer values
train['protocol_type'].replace(['tcp', 'udp','icmp'],
                               [0, 1,2], inplace=True)

train['service'].replace(services,services_num, inplace=True)

train['flag'].replace(flags,flags_num, inplace=True)

train['label'].replace(labels,

```

```

▶ train['label'].replace(labels,
                        labels_num, inplace=True)

test['protocol_type'].replace(['tcp', 'udp','icmp'],
                              [0, 1,2], inplace=True)

test['service'].replace(services,services_num, inplace=True)

test['flag'].replace(flags,flags_num, inplace=True)

test['label'].replace(labels,
                      labels_num, inplace=True)

```



```
print(train.shape)
```

```
(494021, 42)
```

```
[102] #shuffle the rows of train DataFrame randomly by calling the sample function with frac=1
      #avoid any order or bias in the training data
      train = train.sample(frac=1).reset_index()
      #remove first column of the shuffled train dataframe (old index column)
      train=train.iloc[0:12500,1:]
```

```
[103] print(train.shape)
```

```
(12500, 42)
```

```
[104] def kmeans(train, k, max_iters=100, num_restarts=5):
      # tot: number of total samples in train dataset
      # feat: number of features or columns in train dataset
      tot,feat=train.shape
      tot=tot+1
      # print shape of input training set
      print(train.shape)
```

```
best_centroids = None
best_clusters=None
best_conf=None
best_loss = float('inf')

for i in range(num_restarts):
    # Initialize centroids as a k x feat matrix of zeros
    centroids = np.zeros((k, feat))
    # Initialize a confusion matrix as a (tot+1) x 2 matrix of zeros
    conf = np.zeros((tot, 2))
    # For each centroid, choose a random sample from the training set
    for j in range(k):
        centroids[j] = train.sample().values
    # Run k-means algorithm for a maximum of max_iters iterations
    for _ in range(max_iters):
        # Initialize an empty list of clusters
        clusters = [[] for _ in range(k)]
        # find the closest centroid and assign it to the corresponding cluster
        for sample_idx, sample in train.iterrows():
            centroid_idx = np.argmin([np.sqrt(np.sum((sample[0:-1] - centroid[0:-1])**2)) for centroid in centroids])
            clusters[centroid_idx].append(sample_idx)
        # Update the confusion matrix with the assigned centroid index and the actual label
        conf[sample_idx,0]=centroid_idx
        conf[sample_idx,1]=sample['label']

        oldc=centroids
        # Compute the new centroid by taking the mean of the samples in the cluster
        for m, cluster in enumerate(clusters):
            if cluster:
                centroids[m] = train.loc[cluster].mean().values
```

```

# Calculate the total sum of squared distances between each sample and its assigned centroid
loss = sum(sum([np.sum((train.loc[cluster] - centroids[j])**2) for j, cluster in enumerate(clusters)]))
print(loss)
print(best_loss)
print(centroids[0])
print(olddc[0])
if loss < best_loss:
    best_loss = loss
    best_centroids = centroids
    best_clusters=clusters
    best_conf=conf

return best_centroids, best_clusters, best_conf

```

```

def makeconftrain(clusters,confusion,centroids,k):
    # creates a matrix of zeros with k rows (one for each cluster) and 40 columns (one for each class in the dataset)
    matching=np.zeros((k,40))
    # iterates over each cluster in the clusters list, and assigns a unique index i to each cluster
    for i, cluster in enumerate(clusters):
        # iterates over each sample in the current cluster
        for sample in cluster:
            # increments the count for the class of the current sample in the matching matrix, for the current cluster
            matching[i,int(confusion[sample,1])]=matching[i,int(confusion[sample,1])]+1
    # creates an array of -1 values with length k, which will be used to store the predicted labels for each cluster
    cllabel=np.ones(k)*-1
    #iterates over each cluster, and assigns the index i to each one
    for i, cluster in enumerate(clusters):
        # set the last element of ith row of centroids matrix to the index of the highest count class in the matching matrix for the ith cluster.
        centroids[i,-1]=np.argmax(matching[i,:])
        # sets the ith element of cllabel to the index of the highest count class in the matching matrix for the ith cluster.
        cllabel[i]=np.argmax(matching[i,:])
        # creates a 40x40 matrix of zeros, which will be used to store the confusion matrix for the training set
    trainconfusion=np.zeros((40,40))
    for sample in confusion:
        # increments the count in the confusion matrix for the true label of the sample and the predicted label of the sample
        # by using the true label as the row index and the predicted label as the column index of the trainconfusion matrix
        trainconfusion[int(sample[1]),int(cllabel[int(sample[0])])]=trainconfusion[int(sample[1]),int(cllabel[int(sample[0])])]+1
    return trainconfusion,cllabel

```

```

[ ] test = test.sample(frac=1).reset_index()
test=test.iloc[0:10000,1:]

```

```

# initializes an array of zeros named testconf with dimensions equal to the number of samples in the test DataFrame and two columns.
# initializes a list of empty lists named clusters with dimensions equal to the number of centroids in the centroids numpy array

def predict(test, centroids):
    tot,feat=test.shape
    testconf=np.zeros((tot,2))
    clusters = [[] for _ in range(centroids.shape[0])]
    # iterates through each sample in the test DataFrame and determines the index of the nearest centroid using the Euclidean distance formula
    # The index of the nearest centroid is then used to append the sample index to the corresponding list in the clusters list
    # testconf array is updated with the index of the nearest centroid and the label of the sample.
    for sample_idx, sample in test.iterrows():
        centroid_idx = np.argmin([np.sqrt(np.sum((sample[0:-1] - centroid[0:-1])**2)) for centroid in centroids])
        clusters[centroid_idx].append(sample_idx)
        testconf[sample_idx][0]=centroid_idx
        testconf[sample_idx][1]=sample['label']
    return clusters,testconf

```

```

[108] # creates an empty testconfusion matrix of size (40, 40) and fills it with counts of how many test samples belong to
# each cluster and each true class. Finally, the function returns testconfusion
# testconfusion is used to compute the accuracy of the test set predictions. The accuracy is computed as the trace of the testconfusion matrix
# divided by the total number of test samples. The result is printed to the console.
def makeconftest(clusters,testconf,cllabel):
    testconfusion=np.zeros((40,40))
    for sample in testconf:
        testconfusion[int(sample[1]),int(cllabel[int(sample[0])])]=testconfusion[int(sample[1]),int(cllabel[int(sample[0])])]+1
    return testconfusion

```

K = 7

```
0s  # Use the k-means function with the train dataset, 7 clusters  
centroids7,clusters7,confusion7=kmeans(train,7,100,1)
```

```
 (12500, 42)  
59585392934129.58  
inf  
[1.73710753e+00 1.90076460e+00 8.70440865e+00 3.25361965e-04  
3.67105434e+03 8.30304213e+01 0.00000000e+00 4.88042948e-04  
0.00000000e+00 1.80575891e-02 0.00000000e+00 4.79908899e-02  
8.45941109e-03 0.00000000e+00 0.00000000e+00 1.62680983e-04  
1.62680983e-04 3.25361965e-04 3.25361965e-04 0.00000000e+00  
0.00000000e+00 0.00000000e+00 4.84715796e+02 4.84735969e+02  
5.36847243e-05 5.36847243e-05 7.36944851e-04 1.14202050e-03  
9.99455019e-01 7.85749146e-04 1.47470311e-02 2.48998699e+02  
2.48765414e+02 9.81732552e-01 2.91198959e-03 9.54450952e-01  
6.00292826e-04 2.06604848e-04 6.18187734e-05 1.22824142e-03  
6.71872458e-04 4.07922564e+00]  
[1.73710753e+00 1.90076460e+00 8.70440865e+00 3.25361965e-04  
3.67105434e+03 8.30304213e+01 0.00000000e+00 4.88042948e-04  
0.00000000e+00 1.80575891e-02 0.00000000e+00 4.79908899e-02  
8.45941109e-03 0.00000000e+00 0.00000000e+00 1.62680983e-04  
1.62680983e-04 3.25361965e-04 3.25361965e-04 0.00000000e+00  
0.00000000e+00 0.00000000e+00 4.84715796e+02 4.84735969e+02  
5.36847243e-05 5.36847243e-05 7.36944851e-04 1.14202050e-03  
9.99455019e-01 7.85749146e-04 1.47470311e-02 2.48998699e+02  
2.48765414e+02 9.81732552e-01 2.91198959e-03 9.54450952e-01  
6.00292826e-04 2.06604848e-04 6.18187734e-05 1.22824142e-03  
6.71872458e-04 4.07922564e+00]
```

```
[63] # make confusion train matrix of k = 7 clusters  
trainconfusion7,cclabel7=makeconftrain(clusters7,confusion7,centroids7,7)  
print(np.trace(trainconfusion7)/sum(sum(trainconfusion7)))
```

```
0.8528917686585074
```

```
[64] print(clusters7)
```

```
[[0, 1, 2, 3, 6, 9, 16, 17, 18, 20, 21, 22, 23, 25, 26, 27, 28, 29, 30, 31,
```



```
 print(test.shape)  
testclusters7,testconf7=predict(test,centroids7)
```

```
 (10000, 42)
```

```
[66] # make confusion test matrix of k = 7 clusters  
testconfusion7=makeconfptest(clusters7,testconf7,cclabel7)  
print(np.trace(testconfusion7)/sum(sum(testconfusion7)))
```

```
0.4467
```

K = 15

```
# Use the k-means function with the train dataset, 15 clusters
centroids15,clusters15,confusion15=kmeans(train,15,100,1)
```

```
(12500, 42)
59541676351087.766
inf
[0.00000000e+00 2.00000000e+00 9.00000000e+00 0.00000000e+00
 9.59556405e+02 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 5.11000000e+02 5.10998088e+02
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 1.00000000e+00 0.00000000e+00 0.00000000e+00 2.55000000e+02
 2.54931862e+02 9.99732314e-01 4.17173649e-05 9.99732314e-01
 0.00000000e+00 6.95289414e-06 0.00000000e+00 3.47644707e-06
 0.00000000e+00 4.00000000e+00]
[0.00000000e+00 2.00000000e+00 9.00000000e+00 0.00000000e+00
 9.59556405e+02 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 5.11000000e+02 5.10998088e+02
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 1.00000000e+00 0.00000000e+00 0.00000000e+00 2.55000000e+02
 2.54931862e+02 9.99732314e-01 4.17173649e-05 9.99732314e-01
 0.00000000e+00 6.95289414e-06 0.00000000e+00 3.47644707e-06
 0.00000000e+00 4.00000000e+00]
```

```
[ ] # make confusion train matrix of k = 15 clusters
trainconfusion15,cllabel15=makeconftrain(clusters15,confusion15,centroids15,15)
print(np.trace(trainconfusion15)/sum(sum(trainconfusion15)))
```

0.8611311095112391

```
[ ] print(clusters15)
```

[[0, 1, 2, 3, 6, 9, 15, 16, 18, 20, 21, 22, 23, 25, 26, 27, 28, 29, 30, 31, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42]]

```
[ ] print(test.shape)
testclusters15,testconf15=predict(test,centroids15)
```

(10000, 42)

```
# make confusion test matrix of k = 15 clusters
testconfusion15=makeconfptest(clusters15,testconf15,cllabel15)
print(np.trace(testconfusion15)/sum(sum(testconfusion15)))
```

0.4176

K = 23

```
# Use the k-means function with the train dataset, 23 clusters
centroids23,clusters23,confusion23=kmeans(train,23,100,1)
```

```
(12500, 42)
83586631555933.08
inf
[0.00000000e+00 2.00000000e+00 9.00000000e+00 0.00000000e+00
 1.03200000e+03 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 5.10015083e+02 5.09984917e+02
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 1.00000000e+00 0.00000000e+00 0.00000000e+00 2.55000000e+02
 2.55000000e+02 1.00000000e+00 0.00000000e+00 1.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 4.00000000e+00]
[0.00000000e+00 2.00000000e+00 9.00000000e+00 0.00000000e+00
 1.03200000e+03 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 0.00000000e+00 5.10015083e+02 5.09984917e+02
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 1.00000000e+00 0.00000000e+00 0.00000000e+00 2.55000000e+02
 2.55000000e+02 1.00000000e+00 0.00000000e+00 1.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00 0.00000000e+00
 0.00000000e+00 4.00000000e+00]
```

```
[110] # make confusion train matrix of k = 23 clusters
trainconfusion23,clabel23=makeconftrain(clusters23,confusion23,centroids23,23)
print(np.trace(trainconfusion23)/sum(sum(trainconfusion23)))
```

0.9577633789296857

```
[111] print(clusters23)
```

[[16, 19, 25, 29, 36, 73, 106, 121, 129, 146, 148, 160, 178, 180, 185, 186, 192,



```
[112] print(test.shape)
testclusters23,testconf23=predict(test,centroids23)
```

(10000, 42)

```
# make confusion test matrix of k = 23 clusters
testconfusion23=makeconftest(clusters23,testconf23,clabel23)
print(np.trace(testconfusion23)/sum(sum(testconfusion23)))
```



0.6727

K = 31



```
# Use the k-means function with the train dataset, 31 clusters
centroids31,clusters31,confusion31=kmeans(train,31,100,1)
```



(12500, 42)

84070095339493.4

inf

```
[ 0.      2.      9.      0.     520.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.     508.56808511 508.56808511 0.
  0.      0.      0.      1.      0.
  0.     255.     255.      1.      0.
  1.      0.      0.      0.      0.
  0.      4.      ]
[ 0.      2.      9.      0.     520.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.     508.56808511 508.56808511 0.
  0.      0.      0.      1.      0.
  0.     255.     255.      1.      0.
  1.      0.      0.      0.      0.
  0.      4.      ]
```

```
[117] # make confusion train matrix of k = 31 clusters
trainconfusion31,cllabel31=makeconftrain(clusters31,confusion31,centroids31,31)
print(np.trace(trainconfusion31)/sum(sum(trainconfusion31)))
```

0.9428845692344613

```
[118] print(clusters31)
```

[[0, 6, 20, 28, 40, 48, 51, 62, 70, 101, 105, 110, 117, 151, 175, 177, 209, 228,



```
print(test.shape)
testclusters31,testconf31=predict(test,centroids31)
```



(10000, 42)

```
[120] # make confusion test matrix of k = 31 clusters
testconfusion31=makeconfptest(clusters31,testconf31,cllabel31)
print(np.trace(testconfusion31)/sum(sum(testconfusion31)))
```

0.5151

K = 45

```
# Use the k-means function with the train dataset, 45 clusters
centroids45,clusters45,confusion45=kmeans(train,45,100,1)
```

```
(12500, 42)
79947165770056.44
inf
[ 0.      2.      9.      0.     520.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.     507.65602472 507.65602472 0.
  0.      0.      0.      1.      0.
  0.     255.     255.      1.      0.
  1.      0.      0.      0.      0.
  0.      4.      ]
[ 0.      2.      9.      0.     520.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.      0.      0.      0.
  0.      0.     507.65602472 507.65602472 0.
  0.      0.      0.      1.      0.
  0.     255.     255.      1.      0.
  1.      0.      0.      0.      0.
  0.      4.      ]
```

```
[124] # make confusion train matrix of k = 45 clusters
trainconfusion45,cllabel45=makeconftrain(clusters45,confusion45,centroids45,45)
print(np.trace(trainconfusion45)/sum(sum(trainconfusion45)))
```

0.974642028637709

```
print(clusters45)
```

```
[[0, 6, 20, 28, 40, 48, 51, 62, 70, 100, 101, 105, 110, 117, 151, 175, 177, 209, 2
```

```
[126] print(test.shape)
testclusters45,testconf45=predict(test,centroids45)
```

(10000, 42)

```
[127] # make confusion test matrix of k = 45 clusters
testconfusion45=makeconfptest(clusters45,testconf45,cllabel45)
print(np.trace(testconfusion45)/sum(sum(testconfusion45)))
```

0.5228

Normalized Cut

```
[130] # split the train dataset into train2 and test2 sets with a ratio of 0.9975.  
#train2 and test2 contain the feature columns of the train dataset while ytrain2 and ytest2 contain the corresponding labels  
np.random.seed(42)  
random.seed(42)  
random_state = check_random_state(42)  
  
X = train.iloc[:, :-1] # features  
y = train.iloc[:, -1] # labels  
train2, test2, ytrain2, ytest2 = train_test_split(X, y, test_size=0.9975, random_state=random_state)
```

```
[131] print(train2.shape)
```

```
(31, 41)
```

```
▶ # A is the RBF kernel matrix computed from the training data  
A=rbf_kernel(train2)  
# D is the diagonal matrix of the row sums of A  
D=np.diag(np.sum(A,axis=1))  
# L is the graph Laplacian matrix  
L=D-A  
# Dinv is the inverse of D  
Dinv=np.linalg.inv(D)  
# L_norm is the normalized graph Laplacian  
L_norm = Dinv @ L
```

```
[133] # Compute the eigenvectors of the Laplacian matrix using sparse eigen decomposition  
_, X = eigs(L_norm, k=23, which="SM")  
# Take the real part of the eigenvectors to remove any imaginary component  
X = np.real(X)  
# Normalize the eigenvectors to unit length  
X = normalize(X, axis=0)  
# Perform k-means clustering with 23 clusters on the normalized eigenvectors  
kmeans = KMeans(n_clusters=23).fit(X)  
# Reshape the cluster assignments to match the shape of the original training labels  
mask = np.reshape(kmeans.labels_, ytrain2.shape)  
# Get the cluster centers as the final embedding  
centers = kmeans.cluster_centers_
```

```
/usr/local/lib/python3.9/dist-packages/sklearn/cluster/_kmeans.py:870: FutureWarning:  
warnings.warn(  
◀
```

```
▶ print(X.shape)
```

```
☞ (31, 23)
```

```
[135] # normalise eigen vectors  
ncutconf=np.zeros((mask.shape[0],2))  
for i in range(mask.shape[0]):  
    ncutconf[i][0]=mask[i]  
    ncutconf[i][1]=ytrain2.iloc[i]
```

```

#computes the confusion matrix for the Ncut clustering method

# Initialize the matching matrix with zeros
matching=np.zeros((23,40))

# Loop through each sample in the Ncut confusion matrix and update the matching matrix
for sample in ncutconf:
    matching[int(sample[0]),int(sample[1])]= matching[int(sample[0]),int(sample[1])] +1

# Find the cluster label for each cluster
# by finding the index of the maximum value in the corresponding row of the matching matrix
clabel=np.ones(23)*-1
for i in range(23):
    clabel[i]=np.argmax(matching[i,:])

# Compute the confusion matrix for the training data using the computed cluster labels and the ground truth labels
trainconfusionncut=np.zeros((40,40))
for sample in ncutconf:
    trainconfusionncut[int(sample[1]),int(clabel[int(sample[0])])]=trainconfusionncut[int(sample[1]),int(clabel[int(sample[0])])]+1
# Compute the accuracy by dividing the trace of the confusion matrix by the sum of all its elements
print(np.trace(trainconfusionncut)/sum(sum(trainconfusionncut)))

```

0.967741935483871

Kmeans Evaluation Using Train Dataset

```
#Compute Precision, Recall, and F1 score
def precision_recall_f1_train(trainconfusion):
    eps=1e-9
    tp = np.diagonal(trainconfusion)
    fp = np.sum(trainconfusion, axis=0) - tp
    fn = np.sum(trainconfusion, axis=1) - tp
    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)
    f1_score = 2 * (precision * recall) / (precision + recall + eps)
    return precision, recall, f1_score

#Create a function to compute the Conditional Entropy
def conditional_entropy_train(trainconfusion):
    eps = 1e-10
    # Normalize the confusion matrix
    norm_confusion_matrix = trainconfusion / (np.sum(trainconfusion, axis=1, keepdims=True) + eps)
    norm_confusion_matrix += eps
    # Compute the conditional entropy
    entropy = -np.sum(np.nan_to_num(norm_confusion_matrix * np.log2(norm_confusion_matrix), neginf=0), axis=1)
    # Compute the overall conditional entropy
    labels_count = np.sum(trainconfusion, axis=1)
    total_samples = np.sum(labels_count)
    overall_cond_entropy = np.sum(entropy * labels_count) / total_samples

    return entropy, overall_cond_entropy
```

```
# K = 7
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision, recall, f1_score = precision_recall_f1_train(trainconfusion7)
cond_entropy, overall_cond_entropy_train = conditional_entropy_train(trainconfusion7)

# Get the overall Precision, Recall, and F1 score
labels_count_train = np.sum(trainconfusion7, axis=1)
total_samples_train = np.sum(labels_count_train)

# Weighted average
weighted_precision_train = np.sum(precision * labels_count_train) / total_samples_train
weighted_recall_train = np.sum(recall * labels_count_train) / total_samples_train
weighted_f1_score_train = np.sum(f1_score * labels_count_train) / total_samples_train
#weighted_cond_entropy = np.sum(cond_entropy * labels_count) / total_samples

print("K = 7")
print("Precision:", weighted_precision_train)
print("Recall:", weighted_recall_train)
print("F1 Score:", weighted_f1_score_train)
print("Conditional Entropy:", overall_cond_entropy_train)
```

```
➤ K = 7
Precision: 0.8753485585380245
Recall: 0.8528917686583205
F1 Score: 0.8245499513497646
Conditional Entropy: 0.27351935786169007
```

```

▶ # K = 15
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision, recall, f1_score = precision_recall_f1_train(trainconfusion15)
cond_entropy, overall_cond_entropy_train = conditional_entropy_train(trainconfusion15)

# Get the overall Precision, Recall, and F1 score
labels_count_train = np.sum(trainconfusion15, axis=1)
total_samples_train = np.sum(labels_count_train)

# Weighted average
weighted_precision_train = np.sum(precision * labels_count_train) / total_samples_train
weighted_recall_train = np.sum(recall * labels_count_train) / total_samples_train
weighted_f1_score_train = np.sum(f1_score * labels_count_train) / total_samples_train
#weighted_cond_entropy = np.sum(cond_entropy * labels_count) / total_samples

print("K = 15")
print("Precision:", weighted_precision_train)
print("Recall:", weighted_recall_train)
print("F1 Score:", weighted_f1_score_train)
print("Conditional Entropy:", overall_cond_entropy_train)

```

```

↳ K = 15
Precision: 0.8838379094741835
Recall: 0.8611311095109799
F1 Score: 0.836424159705755
Conditional Entropy: 0.2739461317475885

```

```

▶ # K = 23
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision, recall, f1_score = precision_recall_f1_train(trainconfusion23)
cond_entropy, overall_cond_entropy_train = conditional_entropy_train(trainconfusion23)

# Get the overall Precision, Recall, and F1 score
labels_count_train = np.sum(trainconfusion15, axis=1)
total_samples_train = np.sum(labels_count_train)

# Weighted average
weighted_precision_train = np.sum(precision * labels_count_train) / total_samples_train
weighted_recall_train = np.sum(recall * labels_count_train) / total_samples_train
weighted_f1_score_train = np.sum(f1_score * labels_count_train) / total_samples_train
#weighted_cond_entropy = np.sum(cond_entropy * labels_count) / total_samples

print("K = 23")
print("Precision:", weighted_precision_train)
print("Recall:", weighted_recall_train)
print("F1 Score:", weighted_f1_score_train)
print("Conditional Entropy:", overall_cond_entropy_train)

```

```

↳ K = 23
Precision: 0.9398543372386294
Recall: 0.9569330624175901
F1 Score: 0.9482006514193213
Conditional Entropy: 0.1448964873175516

```

```

▶ # K = 31
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision, recall, f1_score = precision_recall_f1_train(trainconfusion31)
cond_entropy, overall_cond_entropy_train = conditional_entropy_train(trainconfusion31)

# Get the overall Precision, Recall, and F1 score
labels_count_train = np.sum(trainconfusion31, axis=1)
total_samples_train = np.sum(labels_count_train)

# Weighted average
weighted_precision_train = np.sum(precision * labels_count_train) / total_samples_train
weighted_recall_train = np.sum(recall * labels_count_train) / total_samples_train
weighted_f1_score_train = np.sum(f1_score * labels_count_train) / total_samples_train
#weighted_cond_entropy = np.sum(cond_entropy * labels_count) / total_samples

print("K = 31")
print("Precision:", weighted_precision_train)
print("Recall:", weighted_recall_train)
print("F1 Score:", weighted_f1_score_train)
print("Conditional Entropy:", overall_cond_entropy_train)

```

```

↳ K = 31
Precision: 0.9282542759052373
Recall: 0.9428845692342376
F1 Score: 0.932946568686486
Conditional Entropy: 0.18497176345029206

```

```

▶ # K = 45
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision, recall, f1_score = precision_recall_f1_train(trainconfusion45)
cond_entropy, overall_cond_entropy_train = conditional_entropy_train(trainconfusion45)

# Get the overall Precision, Recall, and F1 score
labels_count_train = np.sum(trainconfusion45, axis=1)
total_samples_train = np.sum(labels_count_train)

# Weighted average
weighted_precision_train = np.sum(precision * labels_count_train) / total_samples_train
weighted_recall_train = np.sum(recall * labels_count_train) / total_samples_train
weighted_f1_score_train = np.sum(f1_score * labels_count_train) / total_samples_train
#weighted_cond_entropy = np.sum(cond_entropy * labels_count) / total_samples

print("K = 45")
print("Precision:", weighted_precision_train)
print("Recall:", weighted_recall_train)
print("F1 Score:", weighted_f1_score_train)
print("Conditional Entropy:", overall_cond_entropy_train)

```

```

↳ K = 45
Precision: 0.9645692850955185
Recall: 0.9746420286373932
F1 Score: 0.9690723582552736
Conditional Entropy: 0.09797305393375313

```

Kmeans Evaluation Using Test Dataset

```
#Compute Precision, Recall, and F1 score
def precision_recall_f1_test(testconfusion):
    eps=1e-9
    tp = np.diagonal(testconfusion)
    fp = np.sum(testconfusion, axis=0) - tp
    fn = np.sum(testconfusion, axis=1) - tp
    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)
    f1_score = 2 * (precision * recall) / (precision + recall + eps)
    return precision, recall, f1_score

#Create a function to compute the Conditional Entropy
def conditional_entropy_test(testconfusion):
    eps = 1e-10
    # Normalize the confusion matrix
    norm_confusion_matrix = testconfusion / (np.sum(testconfusion, axis=1, keepdims=True) + eps)
    norm_confusion_matrix += eps
    # Compute the conditional entropy
    entropy = -np.sum(np.nan_to_num(norm_confusion_matrix * np.log2(norm_confusion_matrix), neginf=0), axis=1)
    # Compute the overall conditional entropy
    labels_count = np.sum(testconfusion, axis=1)
    total_samples = np.sum(labels_count)
    overall_cond_entropy = np.sum(entropy * labels_count) / total_samples

    return entropy, overall_cond_entropy
```

```
# K = 7

#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision_test, recall_test, f1_score_test = precision_recall_f1_test(testconfusion7)
cond_entropy_test, overall_cond_entropy_test = conditional_entropy_test(testconfusion7)

# Get the overall Precision, Recall, and F1 score
labels_count_test = np.sum(testconfusion7, axis=1)
total_samples_test = np.sum(labels_count_test)

# Weighted average
weighted_precision_test = np.sum(precision_test * labels_count_test) / total_samples_test
weighted_recall_test = np.sum(recall_test * labels_count_test) / total_samples_test
weighted_f1_score_test = np.sum(f1_score_test * labels_count_test) / total_samples_test

print("K = 7")
print("Precision:", weighted_precision_test)
print("Recall:", weighted_recall_test)
print("F1 Score:", weighted_f1_score_test)
print("Conditional Entropy:", overall_cond_entropy_test)
```

```
➤ K = 7
Precision: 0.5902959842837558
Recall: 0.4466999999998263
F1 Score: 0.43769704435247275
Conditional Entropy: 0.720606644033084
```



```
# K = 15
```

```
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
```

```
precision_test, recall_test, f1_score_test = precision_recall_f1_test(testconfusion15)  
cond_entropy_test, overall_cond_entropy_test = conditional_entropy_test(testconfusion15)
```

```
# Get the overall Precision, Recall, and F1 score
```

```
labels_count_test = np.sum(testconfusion15, axis=1)  
total_samples_test = np.sum(labels_count_test)
```

```
# Weighted average
```

```
weighted_precision_test = np.sum(precision_test * labels_count_test) / total_samples_test  
weighted_recall_test = np.sum(recall_test * labels_count_test) / total_samples_test  
weighted_f1_score_test = np.sum(f1_score_test * labels_count_test) / total_samples_test
```

```
print("K = 15")  
print("Precision:", weighted_precision_test)  
print("Recall:", weighted_recall_test)  
print("F1 Score:", weighted_f1_score_test)  
print("Conditional Entropy:", overall_cond_entropy_test)
```



```
K = 15  
Precision: 0.6163838679347801  
Recall: 0.4175999999997613  
F1 Score: 0.4218814275388331  
Conditional Entropy: 0.6871778404859293
```



```
# K = 23
```

```
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
```

```
precision_test, recall_test, f1_score_test = precision_recall_f1_test(testconfusion23)  
cond_entropy_test, overall_cond_entropy_test = conditional_entropy_test(testconfusion23)
```

```
# Get the overall Precision, Recall, and F1 score
```

```
labels_count_test = np.sum(testconfusion23, axis=1)  
total_samples_test = np.sum(labels_count_test)
```

```
# Weighted average
```

```
weighted_precision_test = np.sum(precision_test * labels_count_test) / total_samples_test  
weighted_recall_test = np.sum(recall_test * labels_count_test) / total_samples_test  
weighted_f1_score_test = np.sum(f1_score_test * labels_count_test) / total_samples_test
```

```
print("K = 23")  
print("Precision:", weighted_precision_test)  
print("Recall:", weighted_recall_test)  
print("F1 Score:", weighted_f1_score_test)  
print("Conditional Entropy:", overall_cond_entropy_test)
```



```
K = 23  
Precision: 0.6832020384861281  
Recall: 0.672699999999762  
F1 Score: 0.6517474713250574  
Conditional Entropy: 0.7105107250074697
```



```
# K = 31
```

```
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision_test, recall_test, f1_score_test = precision_recall_f1_test(testconfusion31)
cond_entropy_test, overall_cond_entropy_test = conditional_entropy_test(testconfusion31)

# Get the overall Precision, Recall, and F1 score
labels_count_test = np.sum(testconfusion31, axis=1)
total_samples_test = np.sum(labels_count_test)

# Weighted average
weighted_precision_test = np.sum(precision_test * labels_count_test) / total_samples_test
weighted_recall_test = np.sum(recall_test * labels_count_test) / total_samples_test
weighted_f1_score_test = np.sum(f1_score_test * labels_count_test) / total_samples_test

print("K = 31")
print("Precision:", weighted_precision_test)
print("Recall:", weighted_recall_test)
print("F1 Score:", weighted_f1_score_test)
print("Conditional Entropy:", overall_cond_entropy_test)
```



```
K = 31
Precision: 0.6621665626722625
Recall: 0.5150999999997784
F1 Score: 0.48089427506642535
Conditional Entropy: 0.5219225885909471
```



```
# K = 45
```

```
#Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision_test, recall_test, f1_score_test = precision_recall_f1_test(testconfusion45)
cond_entropy_test, overall_cond_entropy_test = conditional_entropy_test(testconfusion45)

# Get the overall Precision, Recall, and F1 score
labels_count_test = np.sum(testconfusion45, axis=1)
total_samples_test = np.sum(labels_count_test)

# Weighted average
weighted_precision_test = np.sum(precision_test * labels_count_test) / total_samples_test
weighted_recall_test = np.sum(recall_test * labels_count_test) / total_samples_test
weighted_f1_score_test = np.sum(f1_score_test * labels_count_test) / total_samples_test

print("K = 45")
print("Precision:", weighted_precision_test)
print("Recall:", weighted_recall_test)
print("F1 Score:", weighted_f1_score_test)
print("Conditional Entropy:", overall_cond_entropy_test)
```



```
K = 45
Precision: 0.754952084328548
Recall: 0.5227999999997743
F1 Score: 0.492891974162841
Conditional Entropy: 0.46408371527860126
```

Normalized Cut Evaluation

```
# Compute Precision, Recall, and F1 score
def precision_recall_f1(trainconfusionncut):
    eps=1e-9
    tp = np.diagonal(trainconfusionncut)
    fp = np.sum(trainconfusionncut, axis=0) - tp
    fn = np.sum(trainconfusionncut, axis=1) - tp
    precision = tp / (tp + fp + eps)
    recall = tp / (tp + fn + eps)
    f1_score = 2 * (precision * recall) / (precision + recall + eps)
    return precision, recall, f1_score

# Create a function to compute the Conditional Entropy
def conditional_entropy(trainconfusionncut):
    eps = 1e-10
    # Normalize the confusion matrix
    norm_confusion_matrix = trainconfusionncut / (np.sum(trainconfusionncut, axis=1, keepdims=True) + eps)
    norm_confusion_matrix += eps
    # Compute the conditional entropy
    entropy = -np.sum(np.nan_to_num(norm_confusion_matrix * np.log2(norm_confusion_matrix), neginf=0), axis=1)
    # Compute the overall conditional entropy
    labels_count = np.sum(trainconfusionncut, axis=1)
    total_samples = np.sum(labels_count)
    overall_cond_entropy = np.sum(entropy * labels_count) / total_samples

    return entropy, overall_cond_entropy

# Calculate the Precision, Recall, F1 score, and Conditional Entropy
precision, recall, f1_score = precision_recall_f1(trainconfusionncut)
cond_entropy, overall_cond_entropy = conditional_entropy(trainconfusionncut)
```

```
# Get the overall Precision, Recall, and F1 score
labels_count = np.sum(trainconfusionncut, axis=1)
total_samples = np.sum(labels_count)

# Weighted average
weighted_precision = np.sum(precision * labels_count) / total_samples
weighted_recall = np.sum(recall * labels_count) / total_samples
weighted_f1_score = np.sum(f1_score * labels_count) / total_samples

# Print the evaluation metrics
print("Precision:", weighted_precision)
print("Recall:   ", weighted_recall)
print("F1 Score: ", weighted_f1_score)
print("Conditional Entropy:", overall_cond_entropy)
```

```
➞ Precision: 0.9695340500472123
Recall:      0.967741935360215
F1 Score:    0.9667364886912092
Conditional Entropy: 0.12581092009846861
```

New Clustering Algorithm:

DBSCAN is a density-based clustering method, which means it works by detecting areas where points are concentrated and where they are separated by areas that are empty or sparse. Points that are not part of a cluster are labeled as noise (labeled by -1).

The algorithm takes as input a dataset X , a radius parameter ϵ , and a minimum number of samples required to form a cluster (min_samples).

First, it starts by initializing all data points to the "unvisited" state (labeled 0). Next, it iterates over each point and checks if it has been visited or not. If not, the function retrieves its neighboring points within a distance of ϵ using $\text{get_neighbors}(X, i, \epsilon)$. If the number of neighbors is less than min_samples , the point is labeled as noise (-1), and the algorithm moves on to the next unvisited point.

If the number of neighbors is greater than or equal to min_samples , a new cluster is formed, and the point is labeled with a new cluster label (C). The algorithm then expands the cluster by recursively adding new points that are within a distance of ϵ from the current cluster using $\text{expand_cluster}(X, \text{labels}, i, \text{neighbors}, C, \epsilon, \text{min_samples})$.

DBSCAN (New Clustering)

```
[76] import pandas as pd
      from sklearn.metrics.pairwise import pairwise_distances
      import numpy as np
      from time import time
      import matplotlib.pyplot as plt
      from sklearn.model_selection import train_test_split
      from sklearn.cluster import KMeans
      import random
      from sklearn.utils import check_random_state
```

```
col_names = ["duration", "protocol_type", "service", "flag", "src_bytes",
             "dst_bytes", "land", "wrong_fragment", "urgent", "hot", "num_failed_logins",
             "logged_in", "num_compromised", "root_shell", "su_attempted", "num_root",
             "num_file_creations", "num_shells", "num_access_files", "num_outbound_cmds",
             "is_host_login", "is_guest_login", "count", "srv_count", "error_rate",
             "srv_error_rate", "error_rate", "srv_error_rate", "same_srv_rate",
             "diff_srv_rate", "srv_diff_host_rate", "dst_host_count", "dst_host_srv_count",
             "dst_host_same_srv_rate", "dst_host_diff_srv_rate", "dst_host_same_src_port_rate",
             "dst_host_srv_diff_host_rate", "dst_host_error_rate", "dst_host_srv_error_rate",
             "dst_host_error_rate", "dst_host_srv_error_rate", "label"]
train = pd.read_csv("/content/drive/MyDrive/PR/assignment2/kddcup.data_10_percent.gz", compression='gzip', header=None, names = col_names)
```

```
[78] train=pd.DataFrame(train).sample(frac=0.025).reset_index(drop=True)
```

```
[79] print(train.shape)
```

(12351, 42)

```
# labeltrain contains all the unique labels in the 'train' dataframe
labeltrain=pd.concat([train['label']]).unique()
labeltrain_num=np.zeros(len(labeltrain))
# for loop replaces each unique label with an integer label.
for i in range (len(labeltrain)):
    labeltrain_num[i]=i

train['label'].replace(labeltrain,
                      labeltrain_num, inplace=True)

# services contains all the unique services in the 'train' dataframe.
# for loop replaces each unique service with an integer service.
services=pd.concat([train['service']]).unique()
services_num=np.zeros(len(services))
for i in range (len(services)):
    services_num[i]=i

# flags contains all the unique flags in the 'train' dataframe.
# for loop replaces each unique flag with an integer flag.
flags=pd.concat([train['flag']]).unique()
flags_num=np.zeros(len(flags))

for i in range(len(flags)):
    flags_num[i]=i

# labels contains all the unique labels in the 'train' dataframe.
# for loop replaces each unique label with an integer label.

labels=pd.concat([train['label']]).unique()
labels_num=np.zeros(len(labels))
for i in range (len(labels)):
    labels_num[i]=i
```

```
[138] train['protocol_type'].replace(['tcp', 'udp', 'icmp'],
                                     [0, 1, 2], inplace=True)

train['service'].replace(services, services_num, inplace=True)

train['flag'].replace(flags, flags_num, inplace=True)
```

```
import numpy as np

def dbscan(X, eps, min_samples):
    labels = np.zeros(len(X))
    C = 0
    for i in range(len(X)):
        if labels[i] != 0:
            continue
        neighbors = get_neighbors(X, i, eps)
        if len(neighbors) < min_samples:
            labels[i] = -1
            continue
        C += 1
        labels[i] = C
        expand_cluster(X, labels, i, neighbors, C, eps, min_samples)
    return labels

def expand_cluster(X, labels, i, neighbors, C, eps, min_samples):
    for j in neighbors:
        if labels[j] == -1:
            labels[j] = C
        elif labels[j] != 0:
            continue
        labels[j] = C
        new_neighbors = get_neighbors(X, j, eps)
```

```
new_neighbors = get_neighbors(X, j, eps)
[139] if len(new_neighbors) >= min_samples:
        neighbors = neighbors.union(new_neighbors)
    return
```

```
def get_neighbors(X, i, eps):
    neighbors = set()
    for j in range(len(X)):
        if np.linalg.norm(X[i] - X[j]) < eps:
            neighbors.add(j)
    return neighbors
```

```
from sklearn.preprocessing import StandardScaler

# create an instance of this class called scaler
scaler = StandardScaler()
# The train DataFrame is copied to train_10. Then, the label column is dropped from train_10
# and stored in a separate variable called label_true
label_true=train['label']
train_10=train.copy()
train_10.drop('label', axis=1, inplace=True)
# the fit_transform() method is used to scale the data in train_10 and store the scaled data
# in a new DataFrame called train_10_scaled. The columns of train_10_scaled are set to be the same
# as the columns of train_10
train_10_scaled = pd.DataFrame(scaler.fit_transform(train_10), columns=train_10.columns)
```

```
[144] X = train_10_scaled.to_numpy()
labels = dbscan(X, eps=2, min_samples=5)
```

```
[ ] # C is a 2D array
# one column containing the predicted labels and another column containing the true labels
# The first column is populated with the predicted labels obtained from a clustering algorithm
# and the second column is populated with the true labels of the data points
c=np.zeros((labels.size,2))
c[:,0]=labels
c[:,1]=label_true
print(c.size)
print(c)
```

```
24702
```

```
[[1. 0.]
 [2. 1.]
 [3. 2.]
 ...
 [3. 2.]
 [3. 2.]
 [2. 1.]]
```

```
▶ # creates a set of unique labels assigned by the clustering algorithm to different data points.
unique_labels = set(labels)
# initializes a 2D array of zeros to store the matching results between true labels and clustering labels
matching=np.zeros ((len(unique_labels),len(labeltrain)))
# populates the matching array with the count of how many times a true label appears in a given cluster label
for sample in c:
    matching [int (sample [0]),int (sample [1])]= matching [int (sample [0]),int (sample[1])]+1

# initializes an array of -1 to store the final assigned labels for each cluster.
cllabel=np.ones (len(unique_labels))*-1
# assigns the final label to each cluster based on the maximum count of true labels assigned to the cluster
for i in range (len(unique_labels)):
    cllabel[i]=np.argmax(matching[i,:])
```

```
[ ] # initializes a 2D array of zeros to store the confusion matrix for the clustering algorithm.
trainconfusion=np.zeros ( (labels.size,labels.size))
# populates the confusion matrix with the count of data points that belong to each true label and the clustering label
for sample in c:
    trainconfusion [int (sample [1]),int(cllabel [int (sample [0])])]=trainconfusion [int (sample [1]),int (cllabel[int (sample[0])])]+1
# computes the clustering accuracy by taking the ratio of the sum of diagonal elements to the total number of data points
print(np.trace(trainconfusion)/sum(sum(trainconfusion)))
```

```
0.9940085823010283
```

```
[ ] #Compute Precision, Recall, and F1 score
def precision_recall_f1(trainconfusion):
    eps=1e-9
    tp = np.diagonal (trainconfusion)
    fp = np. sum(trainconfusion, axis=0) - tp
    fn = np. sum(trainconfusion, axis=1)- tp
    p = tp / (tp + fp + eps)
    r = tp / (tp + fn + eps)
    f1_score = 2 * (p * r) / (p + r + eps)
    return p, r
```

```
[ ] p,r=precision_recall_f1(trainconfusion)
```

```
count=np.sum(trainconfusion,axis=1)
total=np.sum(count)
p=np.sum(p*count)/total
r=np.sum(r*count)/total
print("Precision: {:.3f}".format(p))
print("Recall: {:.3f}".format(r))
```

```
Precision: 0.993
Recall: 0.994
```

```
▶ f1 = 2 * (p * r) / (p + r)
print("f1: {:.3f}".format(f1))
```

```
↳ f1: 0.994
```

```
▶ # Compute the conditional entropy
n = np.sum(trainconfusion)
ce = 0
for i in range(trainconfusion.shape[0]):
    for j in range(trainconfusion.shape[1]):
        if trainconfusion[i,j] > 0:
            p_ij = trainconfusion[i,j] / n
            p_j = np.sum(trainconfusion[:,j]) / n
            ce += p_ij * np.log2(p_j / p_ij)

print("Conditional entropy: {:.3f}".format(ce))
```

```
↳ Conditional entropy: 0.059
```